23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE
Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

3. Database Design Document

Group No : **12**
Team Members :

Roll No	Student Name
AM.SC.U4CSE25212	Dev Narayan U
AM.SC.U4CSE25227	Kashyap S
AM.SC.U4CSE25244	Rishik Raj P
AM.SC.U4CSE25245	Saharsh Baiju

Project Title :

3.1. Detailed description of the project

[clearly describe the scenario so that the entities, attributes, relationships, users and business rules are evident from the description]

O(1) is a web-based ICPC preparation tracker for competitive-programming teams and their mentors. Students practise problems from platforms such as Codeforces and AtCoder, while mentors can create practice sheets, assign problems and monitor student progress.

The system brings problems, contests, submissions, practice sheets, teams, resources and schedules into one place. Students can link their online judge accounts, solve assigned problems and view their progress. Mentors can create sheets, assign them to students, coach teams and add learning resources.

Main Entities

- **University** – stores university name and country.
- **User** – stores student/mentor details such as name, email, role, year and university.
- **JudgeAccount** – stores a user's Codeforces/AtCoder handle, rating and verification time.
- **Problem** – stores problem title, platform, code, rating, URL and editorial availability.
- **Topic** – represents topics such as Graphs, DP and Trees.
- **Contest** – stores contest details such as name, platform, type, start time and duration.
- **Submission** – records a user's submission, verdict, language and submission time.
- **Participation** – stores a user's contest rank and rating change.
- **Sheet** – a mentor-created collection of problems with a level and optional deadline.

- **Team** – stores team name, university, coach and invite code.
- **Resource** – stores learning materials such as articles, videos and tutorials.
- **CalendarEvent** – stores practice sessions, meetings and announcements.
- **Reminder** – stores reminders set by users for calendar events.

Important Relationships and Business Rules

- A user belongs to exactly one university and has exactly one role: student or mentor.
- A user can have at most one judge account for each platform.
- A problem can belong to multiple topics and can appear in multiple contests.
- A contest contains multiple problems, each having a unique label such as A, B or C.
- A submission belongs to one user and one problem. Contest submissions may also belong to a contest.
- A user can have at most one participation record for a contest.
- A mentor creates practice sheets, and a sheet contains ordered problems.
- A sheet can be assigned to multiple students.
- A team has one mentor as coach and can have at most three students. A student can belong to at most one team.
- A resource covers one topic and is added by one user.
- A user can have at most one reminder for an event.
- Values such as solved count, streak, leaderboard rank and problem status are derived, not stored.

3.2. ER Diagram

[entities, attributes, relationships, keys, cardinality and participation have to be accurately represented]

The ER diagram should contain the following entities, attributes and relationships.

Entities and Primary Keys

Entity	Primary Key	Important Attributes
University	`univ_id`	name, country
User	`user_id`	name, email, password_hash, role, year_of_study, joined_at, bio, avatar_url
JudgeAccount	`user_id + platform`	handle, rating, verified_at
Topic	`topic_id`	name
Problem	`problem_id`	platform, external_code, title, rating, url, has_editorial
Contest	`contest_id`	platform, external_id, name, kind, starts_at, duration_min
Submission	`submission_id`	verdict, language, submitted_at, platform, external_submission_id

Entity	Primary Key	Important Attributes
Participation	`user_id + contest_id`	old_rating, new_rating, rank
Sheet	`sheet_id`	title, description, level, deadline, created_at
Team	`team_id`	name, invite_code, created_at
Resource	`resource_id`	title, type, url, level, duration_min
CalendarEvent	`event_id`	title, kind, starts_at, duration_min
Reminder	`user_id + event_id`	minutes_before

Relationships and Cardinality

Relationship	Cardinality
User — University	N : 1
Team — University	N : 1
Mentor — Team (coaches)	1 : N
User — Team	N : 1, maximum 3 students/team
User — JudgeAccount	1 : N
Problem — Topic	M : N
Contest — Problem	M : N
User — Submission	1 : N
Submission — Problem	N : 1
Submission — Contest	N : 1, optional
User — Contest (Participation)	M : N
Mentor — Sheet	1 : N
Sheet — Problem	M : N
Sheet — User (Assignment)	M : N
User — Sheet — Problem (Completion)	Ternary
Sheet — Contest (Archives)	N : 1, optional
Resource — Topic	N : 1
User — Resource	1 : N
User — Resource (Bookmark)	M : N

Relationship	Cardinality
Team — CalendarEvent	1 : N, optional
User — CalendarEvent (Reminder)	M : N

The ER model uses JudgeAccount as a weak entity, with (user_id, platform) as its key. M:N relationships are converted into separate relations, while 1:N relationships place the key of the 1-side as a foreign key on the N-side.

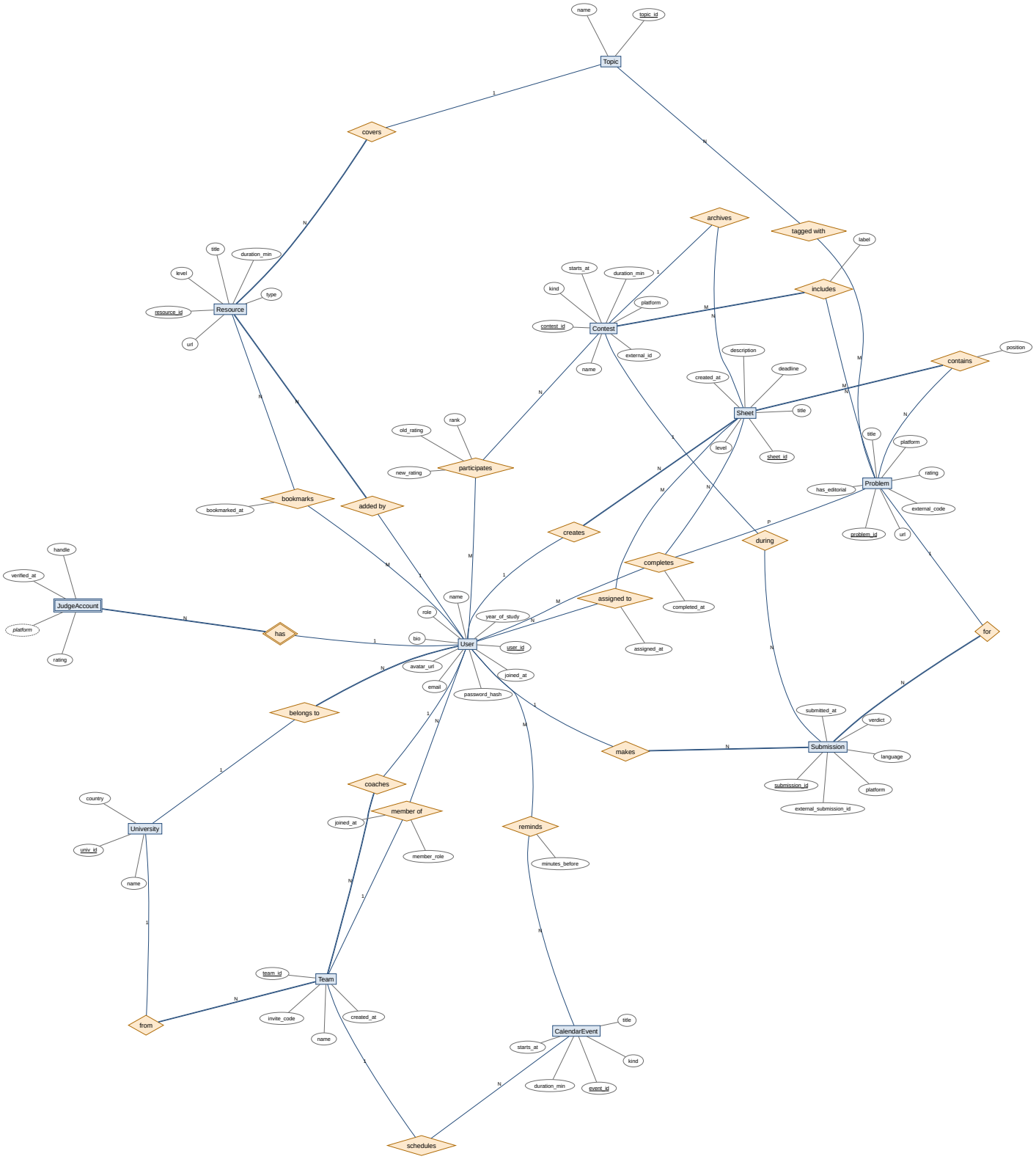


Figure 1 — ER diagram (Chen notation). Rectangles are entity sets, double rectangles weak entity sets, diamonds relationship sets and double diamonds identifying relationships. Ellipses are attributes; an underlined attribute is a key and a dashed italic ellipse is a partial key. Double lines denote total participation, single lines partial participation.

3.3 Relational Schema

[Correctly converts all entities, relationships, cardinality etc to relational schema]

```
University(
    univ_id PK,
    name,
    country
)

Topic(
    topic_id PK,
    name
)

User(
    user_id PK,
    name,
    email UNIQUE,
    password_hash,
    role,
    year_of_study,
    univ_id FK,
    joined_at,
    bio,
    avatar_url
)

JudgeAccount(
    user_id PK, FK,
    platform PK,
    handle,
    rating,
    verified_at
)

Problem(
    problem_id PK,
    platform,
    external_code,
    title,
    rating,
    url,
    has_editorial
)

ProblemTopic(
    problem_id PK, FK,
    topic_id PK, FK
)
```

```
Contest(
    contest_id PK,
    platform,
    external_id,
    name,
    kind,
    starts_at,
    duration_min
)

ContestProblem(
    contest_id PK, FK,
    problem_id PK, FK,
    label
)

Submission(
    submission_id PK,
    user_id FK,
    problem_id FK,
    contest_id FK,
    verdict,
    language,
    submitted_at,
    platform,
    external_submission_id
)

Participation(
    user_id PK, FK,
    contest_id PK, FK,
    old_rating,
    new_rating,
    rank
)

Sheet(
    sheet_id PK,
    title,
    description,
    author_id FK,
    level,
    deadline,
    created_at,
    archive_contest_id FK
)
```

SheetProblem(
    sheet_id PK, FK,
    problem_id PK, FK,
    position
)

SheetAssignment(
    sheet_id PK, FK,
    user_id PK, FK,
    assigned_at
)

SheetCompletion(
    sheet_id PK, FK,
    user_id PK, FK,
    problem_id PK, FK,
    completed_at
)

Team(
    team_id PK,
    name,
    univ_id FK,
    coach_id FK,
    invite_code,
    created_at
)

TeamMember(
    user_id PK, FK,
    team_id FK,
    member_role,
    joined_at
)

Resource(
    resource_id PK,
    title,
    type,
    url,
    topic_id FK,
    level,
    duration_min,
    added_by FK
)

Bookmark(
    user_id PK, FK,
    resource_id PK, FK,
    bookmarked_at
)

CalendarEvent(
    event_id PK,
    title,
    kind,
    starts_at,
    duration_min,
    team_id FK,
    created_by FK
)

Reminder(
    user_id PK, FK,
    event_id PK, FK,
    minutes_before
)

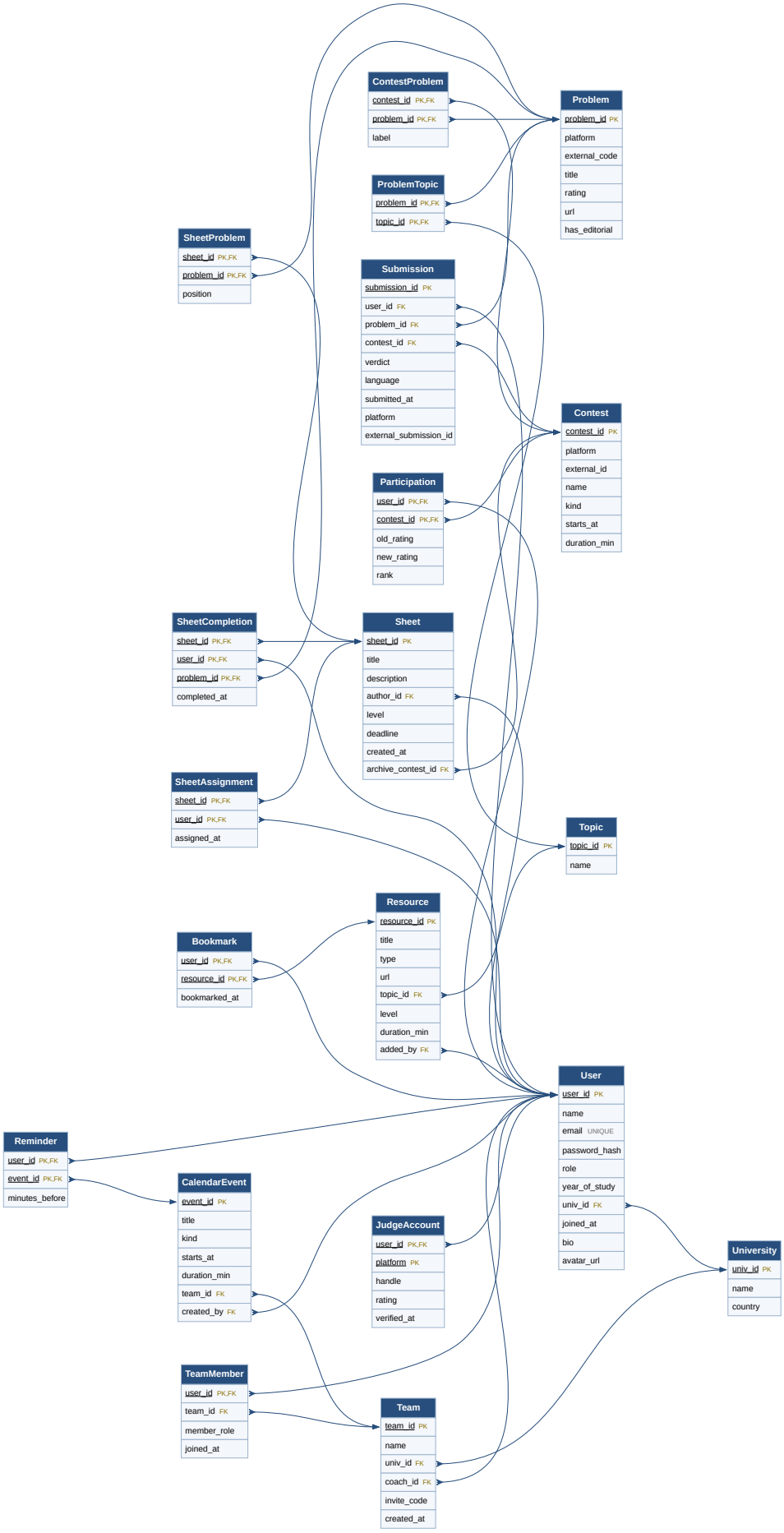


Figure 2 — Relational schema diagram. Each box is a relation; underlined attributes form the primary key, attributes marked FK are foreign keys, and every arrow points from a foreign key to the primary key it references.

3.4 Normalization and Functional dependency analysis

[Correctly identifies relevant FDs and explains their significance

Justify the selection of candidate keys with attribute closure

Demonstrate the normalization process starting from 1NF to BCNF or higher with proper justification]

Functional Dependencies:

The main functional dependencies identified in the database are:

- **University:** `univ_id → name, country`
- **User:** `user_id → name, email, role, year_of_study, univ_id, joined_at, bio, avatar_url`
- **JudgeAccount:** `(user_id, platform) → handle, rating, verified_at`
- **Problem:** `problem_id → platform, external_code, title, rating, url, has_editorial`
- **Contest:** `contest_id → platform, external_id, name, kind, starts_at, duration_min`
- **Submission:** `submission_id → user_id, problem_id, contest_id, verdict, language, submitted_at, platform, external_submission_id`
- **Participation:** `(user_id, contest_id) → old_rating, new_rating, rank`
- **Sheet:** `sheet_id → title, description, author_id, level, deadline, created_at`
- **Team:** `team_id → name, univ_id, coach_id, invite_code, created_at`
- **Resource:** `resource_id → title, type, url, topic_id, level, duration_min, added_by`

These functional dependencies help identify candidate keys and reduce data redundancy.

Candidate Keys using Attribute Closure:

For the Problem relation:

```
problem_id → all other attributes
```

Therefore,

```
{problem_id}+ = all attributes
```

Hence, problem_id is a candidate key.

Also,

```
(platform, external_code) → problem_id
```

Therefore,

```
{platform, external_code}+ = all attributes
```

Hence, (platform, external_code) is also a candidate key.

Therefore, the candidate keys of Problem are:

```
{problem_id} and {platform, external_code}
```

The primary key selected is problem_id.

Normalization:

1NF: All attributes are atomic and repeating values are separated. For example, multiple topics of a problem are stored separately in the ProblemTopic relation.

2NF: Partial dependencies are removed. For example:

```
sheet_id → sheet_title student_id → student_name problem_id →
problem_title
```

These attributes depend only on part of the composite key, so they are separated into their respective relations such as Sheet, User and Problem.

3NF: Transitive dependencies are removed. Examples are:

```
student_id → univ_id → univ_name sheet_id → mentor_id → mentor_name
problem_id → rating → difficulty
```

These dependencies are separated into appropriate relations. Difficulty is derived from rating and is not stored separately.

BCNF: After decomposition, every non-trivial functional dependency has a candidate key as its determinant. Therefore, all the final relations satisfy BCNF. The decomposition is lossless and dependency-preserving.

Thus, the database is normalized as:

1NF → 2NF → 3NF → BCNF.